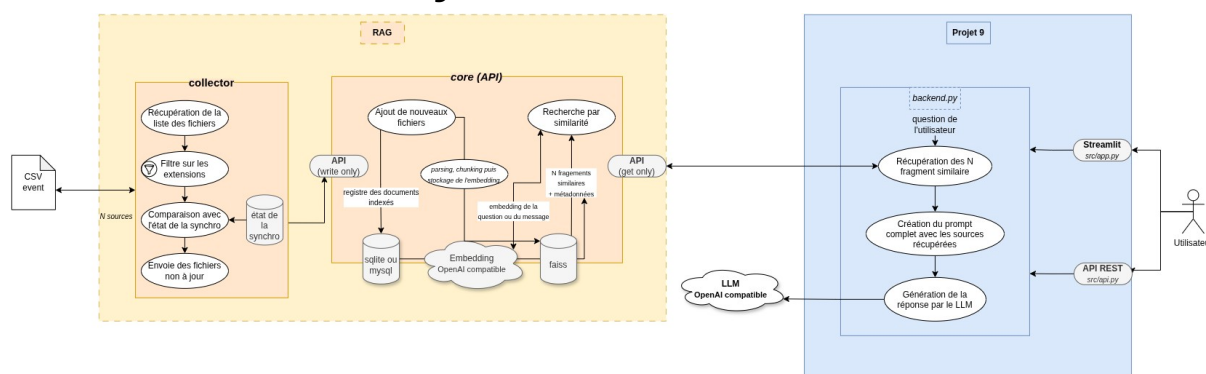


Rapport technique – Assistant intelligent de recommandation d'événements culturels

1. Objectifs du projet

- **Contexte** : mission freelance pour Puls-Events, qui développe une plateforme de recommandations culturelles personnalisées. Le responsable technique (Jérémy) demande un POC de chatbot RAG interrogeant les événements Open Agenda, avec API exploitable par les équipes produit et marketing, démo live conteneurisée et présentation PowerPoint.
- **Problématique** : les questions utilisateurs sont en langage naturel alors que les données sont des fiches structurées. Le RAG permet de combiner recherche vectorielle et génération de la réponse en langage naturel.
- **Objectif du POC** : démontrer la faisabilité technique (LangChain + Mistral + Faiss), la pertinence métier (réponses utiles aux équipes produit/marketing) et la performance (qualité mesurée sur jeu annoté).
- **Périmètre** : ville de Marseille uniquement, 4 715 événements du 04/09/2025 au 09/12/2027 (events_propres.csv).

2. Architecture du système



- Le système suit un pipeline simple. Les événements sont récupérés puis nettoyés. Ils sont ensuite indexés pour la recherche. Le chatbot interroge cet index et rédige la réponse. Tout tourne dans un seul conteneur.
 - Les données viennent d'Open Agenda. Seule Marseille est retenue (choix arbitraire laissé libre). La période va de septembre 2025 à décembre 2027.

- Chaque événement est nettoyé. Son texte est converti en vecteur par le modèle Mistral. Les vecteurs sont stockés dans une base Faiss.
- À chaque question, le système retrouve les événements les plus proches en sens. Puis le modèle de langage rédige la réponse. Il ne s'appuie que sur ces événements.
- Le chatbot s'utilise de deux façons. Une API répond aux questions par HTTP. Une interface web permet de tester à la main.
- **Côté technique** : Python, LangChain, Mistral, Faiss, FastAPI et Docker. Les tests tournent sous pytest.

3. Préparation et vectorisation des données

- **Un script interroge Open Agenda** page par page. Il ne garde que Marseille. Il filtre aussi les dates. Résultat : 4 715 événements propres.
- **Nettoyage** : Les adresses sont uniformisées. Seules les colonnes utiles sont gardées. Des études analyses ont cadré ces choix.
- **Chunking** : Chaque événement est indexé d'un bloc. Le découper n'aurait aucun sens. Un concert reste un document entier.
- **Le texte de chaque événement devient un vecteur.** C'est le modèle Mistral qui s'en charge. La ville et les dates restent à part, comme filtres.
 - Chaque fiche contient le titre, la date, l'adresse et la description. La description longue est préférée quand elle existe.
 - La taille des vecteurs est fixée par Mistral. L'indexation se fait par lots. Un événement égale un vecteur.

4. Choix du modèle NLP

- **Le modèle prévu est Mistral Small.** Mais son API en ligne bloque les requêtes. Pour le développement, la génération passe par un modèle hébergé localement. Le jour où l'API remarche, il suffira de changer la base URL du LLM.
- **Ce choix ne coûte rien et répond vite.** La clé Mistral ne sert qu'aux embeddings (API fonctionnelle avec limite de requête par minute). Le code permet de basculer d'un hébergeur à l'autre.
- **Prompt : le modèle reçoit une consigne stricte.** Répondre en français. N'utiliser que les événements fournis. Dire quand il ne trouve rien. La date du jour est ajoutée pour situer les réponses.

5. Construction de la base vectorielle

- **La base vectorielle utilise Faiss.** Elle retrouve les événements proches en sens d'une question. Cinq résultats par défaut.
- **L'index est sauvegardé sur disque.** Relancer le système ne fait pas tout perdre. Des volumes dédiés portent ces données.
 - À chaque démarrage, le fichier des événements est réingéré. L'index se reconstruit seul. Les vecteurs déjà calculés sont réutilisés.
 - Chaque événement garde son identifiant d'origine. On le retrouve tel quel dans l'index.
- **Chaque fiche garde sa date, sa ville et son statut.** La ville filtre la recherche. Les dates affinent après coup.
 - La recherche renvoie texte et métadonnées ensemble. Le chatbot s'en sert pour répondre et citer ses sources.

6. API et endpoints exposés

- **L'API utilise FastAPI.** La logique du chatbot est séparée du code web. L'API ne fait que l'exposer.
- **Deux routes suffisent.** L'une reçoit les questions. L'autre dit si le service est en vie. Une documentation interactive est générée.
 - On envoie une question, avec ville et dates en option. On reçoit une réponse rédigée et ses sources. Une question vide est refusée. Une panne est signalée proprement.
 - Il n'y a pas de bouton de reconstruction. L'index se refait au démarrage du conteneur. C'est un manque assumé du POC.
- **En local, l'API se lance en une commande.**
- **L'API est testée** en simulé par cinq tests intégrés au pipeline CI (tests/test_api.py). Un script à part vérifie le système réel, avec les réponses RAG réels (la contrainte du multidépôt empêche l'intégration au CI) : eval/api_test.py.
- **Les erreurs sont explicites.** Question vide, service injoignable, mauvais mot de passe : chaque cas a son message. Aucune clé secrète ne circule.

7. Évaluation du système

- **Dix questions avec leurs bonnes réponses.** Elles couvrent jeunesse, emploi, concerts, expo et cinéma.

- Les dix questions passent en conditions réelles. Par l'API, comme un vrai usage.
 - Une réponse est bonne si elle dit la même chose. Même sens, mêmes infos.
 - Les 10 questions du dataset contiennent la liste des sources qui permettent d'y répondre le mieux.
- **Deux scripts mesurent la ressemblance** (via model2vec, un embedding léger qui tourne ne local).
 - **Évaluation des sources** (recall RAG) : Le script *eval/evaluate_rag.py* interroge le RAG directement pour chaque question du dataset et compare les uid retournés aux sources attendues.
Résultat : 76 % de rappel (167 sources retrouvées sur 219 attendues). Les questions précises atteignent 100 %. Les écarts viennent du RAG lors de questions générales : certaines sources pertinentes ne remontent pas dans le top_k pour des questions plus larges (art contemporain, BTP, patrimoine).
 - **Évaluation qualitative des réponses avec Ragas** : le script *eval/eval_qualite.py* interroge le chatbot pour chaque question du dataset, récupère la réponse générée ainsi que les contextes utilisés, puis les compare à la réponse humaine annotée (reponse_attendue) à l'aide de quatre métriques jugées par un LLM local.
 On notera que pour cette évaluation, le modèle n'utilise pas de chaîne de raisonnement. Cela peut diminuer la qualité de l'évaluation.
Résultat :
 - Une fidélité au contexte de 0,90 : le modèle reprend très bien les informations présentes dans ses sources, sans halluciner.
 - Le rappel de contexte s'établit à 0,71 : les documents récupérés couvrent environ 71 % des informations de la réponse attendue.
 - La précision de contexte est de 0,38 : les sources pertinentes sont récupérées mais mal classées.
 - **C'est tout à fait correct.** La recherche trouve l'essentiel. Il reste une marge de progression.
 - Il semble que la principale amélioration réside dans le dataset (qui ne liste pas les événements précis attendu) ; et le modèle d'embedding qui est très léger mais perd en précision (deux phrases ayant le même sens ont deux réponses relativement éloignées ~0.8).
 - Il serait également possible d'améliorer la précision de contexte en améliorant le prompt et peut-être en passant sur un modèle de raisonnement.
 - Les résultats obtenus sont donc plutôt encourageants !

8. Recommandations et perspectives

- **Une seule ville couverte** : il faudrait passer à l'échelle et récupérer la totalité des événements.
- **Un modèle local en attendant Mistral** : cela n'est pas parfaitement conforme à la consigne mais a permis de développer le projet.
- **Une API sans mot de passe** : il faudrait ajouter une authentification.

- **Dix questions dans le dataset de test** : un dataset plus large permettrait d'évaluer le rag avec plus de précision et de pertinence.
- **Ajouter un bouton de reconstruction** : le projet étant basé sur un rag développé par ailleurs (par mes soins), il ne permet pas la reconstruction complète du rag via une simple requête API. Cela pourrait être envisagé.

9. Organisation du dépôt GitHub

- Une partie du projet avait été développée par ailleurs dans le cadre de mon entreprise :
 - **ragifix** : le cœur du RAG, qui expose une API qui reçoit des documents, les traite et les stocke sous forme de vecteurs. L'API permet également de faire une recherche.
 - **ragifix-collector** : un projet qui récupère des données provenant de n'importe quelle source, et les envoie à ragifix. Ici, un connecteur csv_events a été créé.
 - Le dossier **datas/** traite les données sources et les vérifie afin de créer un dataset propre au développement du projet (non publié).
- Le cœur du projet réside dans ocrproject9 : il met en œuvre la totalité du projet :

```
ocrproject9/
├── src/                                # Code source du chatbot.
│   ├── backend.py                    # Client pour le RAG. Gère la recherche et la chaîne
│   │   └── LangChain.
│   ├── api.py                        # API FastAPI. Deux routes : /ask et /health.
│   ├── app.py                        # Interface web Streamlit pour tester le chatbot.
│   └── config.py                     # Lecture des variables d'environnement.
├── tests/                            # 37 tests pytest, tous mockés. Tournent en CI sans
│   └── service externe.
├── eval/                             # Outils d'évaluation du RAG et du LLM.
│   ├── dataset.jsonl                # 10 questions avec 219 sources attendues.
│   ├── evaluate_rag.py              # Mesure le recall du RAG : 76% des sources retrouvées.
│   ├── eval_qualite.py              # Mesure la qualité des réponses LLM par similarité.
│   └── api_test.py                  # Vérifie que le RAG répond en conditions réelles.
├── deploy/                           # Scripts de déploiement Docker.
│   ├── Dockerfile                   # Un seul conteneur pour les 4 services.
│   └── entrypoint.sh                # Orchestre le démarrage : ragifix, collector, API,
│   └── Streamlit.
├── datas/                            # Corpus de données et analyses préparatoires.
│   ├── events_propres.csv           # Corpus final : 4 715 événements à Marseille.
│   └── script.py                    # Script de récupération des données depuis Open
├── Agenda.
│   ├── etude_decoupage.md           # Analyse du chunking : pourquoi un événement = un
│   │   └── document.
│   ├── etude_description.py         # Étude sur le choix de la description longue vs
│   │   └── courte.
│   └── etude_longueur.py            # Analyse de la longueur des textes avant indexation.
```

```
|   └─ etude_status.py    # Analyse des statuts d'événements (annulé, reporté,
etc.).
|   └─ docs/              # Documentation du projet.
|   └─ requirements.txt   # Dépendances Python du projet.
|   └─ README.md          # Guide d'utilisation et architecture globale.
```